



RoadVision - Smart Road Rule Driver Assistance System with Voice Alerts

Graduation Project By

Laith Hamaydeh

Nashat Al-Farajat

Supervisor Dr.Jamil Al-Sawwa

Computer Science Department
Faculty of Information & Communications Technology
Tafila Technical University
2025-2026

1. Introduction

Every second behind the wheel, a driver is expected to process a continuous stream of visual information, road markings, pedestrian crossings, speed limits, warnings and respond to each with precision. In an ideal world, no sign goes unnoticed, no regulation is missed, and every decision is made with full awareness of the road ahead. But roads are imperfect, attention falters, and the consequences of a missed sign can be severe. This is the problem that RoadVision aims to solve.

Traffic signs form the silent language of the road. They regulate speed, warn of danger, direct movement, and ultimately keep drivers, passengers, and pedestrians safe. Their role is so fundamental that modern intelligent transportation systems, driver-assistance systems, and autonomous vehicles all depend on the ability to detect and interpret them reliably and in real time. Yet despite how critical this capability is, achieving it consistently in real-world conditions remains one of the most technically demanding challenges in computer vision. Signs are often small and distant, partially hidden behind trees or other vehicles, faded by weather, and subject to extreme shifts in lighting from the blinding mid-day sun to the shadows of an overpass at dusk. A system that fails under any of these conditions is a system that cannot be trusted.

Existing approaches to traffic sign detection have made meaningful progress, particularly with the rise of deep learning, which has dramatically outperformed older methods that relied on color thresholds and fixed geometric shapes. However, most of the research, datasets, and benchmarks that drive this field forward are built around internationally standardized signs - the kind found in Europe and North America. Jordan, like many countries in the region, operates with its own distinct set of road signs, shaped by local standards, Arabic language requirements, and infrastructure realities that no global dataset accounts for. This means that even a state-of-the-art model trained on the world's most popular benchmarks would be largely blind to the actual signs on Jordanian streets. Deploying such a model here would not be an adaptation, it would be a mismatch. RoadVision proposes to address this gap directly. Rather than relying on existing datasets that do not reflect the realities of Jordanian roads, this project introduces a custom-built dataset collected from real traffic scenes across the country, covering 25 distinct traffic sign classes that reflect the signs drivers actually encounter. This dataset forms the foundation of a detection pipeline built on the latest version of YOLO [1], one of the most powerful and efficient real-time object detection architectures available and fine-tuned specifically for the visual characteristics and challenges of local road environments.

Beyond detection, RoadVision is designed to be a complete, integratable driver-assistance solution. When a traffic sign is identified, the intended system will not simply draw a bounding box on a screen,

it will trigger a contextual voice alert, delivering an immediate, spoken notification to the driver. Whether it is a speed limit approaching, a traffic light, a sharp turn ahead, or a pedestrian crossing in the road, the driver will be informed in the moment it matters. The system is planned to interface with a vehicle's front-facing camera, making it a practical addition to real automotive environments rather than a confined lab experiment.

In a country where road safety remains a pressing public concern and driver assistance technologies have yet to be meaningfully adapted to local road environments and sign conventions, RoadVision represents a locally grounded, technically rigorous, and immediately actionable step toward more effective and context-aware driver assistance.

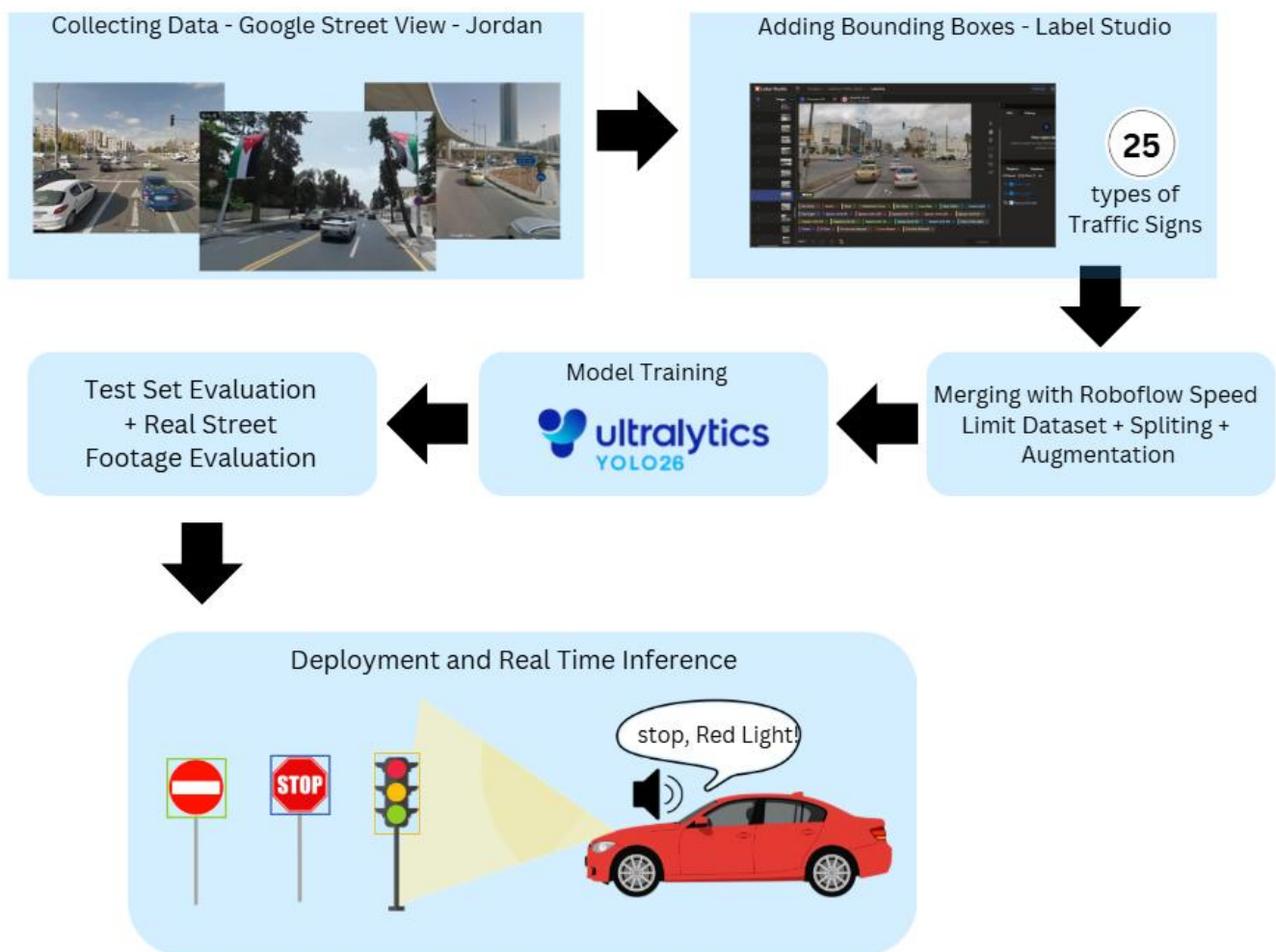


Fig 1: Simplified Project Framework

2. Related Work

Traffic Sign Detection and Recognition (TSDR) has been a significant area of research within computer vision and intelligent transportation systems, attracting considerable attention due to its critical role in autonomous driving, assistance systems and road safety. In [2], an extensive evaluation on two large-scale datasets captured in Belgium and Germany was conducted, demonstrating that existing detection and classification methods without application-specific modifications could achieve accuracy in the range of 95–99%, with top-performing approaches relying on histogram of oriented gradients features for detection and sparse representations for classification. Building on such foundations, more recent work has shifted toward deep learning architectures to address the limitations of traditional feature-based methods.

In [3] You Only Look Once version 5 (YOLOv5) was evaluated against Single Shot Multibox Detector for traffic sign recognition within the context of intelligent transportation systems, finding that the former achieved a mean average precision of 97.70% compared to 90.14%, while also outperforming it in recognition speed. In [4], a DCNN-based system for autonomous traffic and road sign detection and recognition was proposed, along with a newly developed dataset of 2,718 images across 24 traffic sign categories collected from roadsides in Saudi Arabia, named SA-TRS-2018, with their model achieving a recognition accuracy of 100%. In [5], Vision Transformers were investigated to determine whether they could match the performance of convolutional neural networks for traffic sign classification, contributing three open datasets for German, Indian, and Chinese traffic signs, and finding that convolutional neural networks still outperformed transformers by margins of up to 12.81%, while also proposing suggestions to improve transformer performance in this domain.

On the hardware side, [6] proposed a resource-efficient hardware implementation for traffic sign recognition designed for advanced driver assistance systems, combining normalized color transform and connected component labeling for detection with histogram of oriented gradients and support vector machine for classification, achieving a 96.61% detection rate and 90.85% recognition rate on the German Traffic Sign Detection Benchmark (GTSDb) dataset, while operating at 135 frames per second with a power consumption of approximately 8mW.

Addressing the need for richer and more diverse benchmark data, [7] introduced an updated version of the Chinese Traffic Sign Detection Benchmark (CTSDB), expanding it with over 4,000 real-world traffic scene images and detailed annotations, categorized by sign type, size, and weather conditions,

and evaluated nine classic detection algorithms on the new benchmark to guide future research toward more robust and precise detection systems.

In [8], a comprehensive traffic sign recognition system combining image enhancement techniques and (DL) models was presented, demonstrating high accuracy even under complex background conditions and varying environments, with emphasis on its potential integration into autonomous vehicles and advanced driver assistance systems. Similarly focusing on real-world detection challenges, In [9], an enhanced detection algorithm built upon YOLOv8 was proposed, incorporating a small object detection layer and a bidirectional feature pyramid network to improve multi-scale and small object detection, achieving a mean average precision of 90.1% and 78 frames per second on the TT100K dataset, outperforming current mainstream models. In [10], a hybrid detection framework combining YOLO version 12, Detectron2 with Faster R-CNN, and a Vision Transformer classifier was introduced, merging detection outputs through an overlapping box fusion strategy and using the Vision Transformer to verify predictions and filter false positives, achieving 97.7% mean average precision at 0.5 IoU on the Self-Driving Cars Dataset from Roboflow, with notable improvements in detecting small, occluded, and overlapping signs. In [11], a comprehensive survey of traffic sign recognition techniques was presented, motivated by the increasing development of autonomous vehicles and the need for reliable perception systems. The study reviews both conventional machine learning and deep learning-based approaches, covering key components such as preprocessing, feature extraction, classification methods, benchmark datasets, and evaluation metrics. It highlights widely used traffic sign datasets and discusses their limitations in terms of variability, annotation quality, and environmental conditions. The survey identifies major challenges including geographical differences in traffic sign design, complex backgrounds, and illumination variations that affect recognition performance. Finally, it outlines future research directions toward more robust and generalizable traffic sign recognition systems for real-world autonomous driving applications.

Table 1 presents the key contributions and limitations of the previous works.

Table 1: Contributions and Limitations

paper	Key Contributions	Limitations
Mathias et al. (2013) [2]	Large-scale evaluation on Belgian and German datasets; demonstrated HOG-based detection + sparse representation classification can reach 95–99% accuracy without dataset-specific tuning.	Relies (HOG); on hand-crafted features limited generalizability beyond European sign conventions; no real-time performance evaluation or embedded deployment consideration.
Zhu & Yan (2022) [3]	Comparative study of YOLOv5 vs. SSD for traffic sign recognition; YOLOv5 achieved 97.70% mAP and superior speed over SSD (90.14% mAP).	Evaluated on a single dataset without regional diversity; no integration into a real-time driver assistance pipeline or voice alert system; does not consider embedded deployment.
Alghmgham et al. (2019) [4]	Custom Saudi Arabian dataset (SA-TRS-2018) with 2,718 images across 24 categories; DCNN-based system achieving 100% recognition accuracy in a regional context.	100% accuracy may reflect overfitting or a constrained test set; small dataset size (2,718 images); no real-time deployment or driver assistance integration described.
Zheng & Jiang (2022) [5]	ViT vs. CNN comparison for traffic sign classification; three open datasets (German, Indian, Chinese); found CNNs outperform ViTs by up to 12.81% and proposed improvements for transformers.	Evaluation limited to classification only (no detection pipeline); Arab/Middle Eastern road environments not represented; no real-time or embedded system evaluation.
Weng & Chiu (2018) [6]	Resource-efficient FPGA-based ADAS implementation using HOG + SVM; 96.61% detection rate and 90.85% recognition rate at 135 fps with only ~8 mW power consumption on GTSDB.	Traditional HOG+SVM pipeline limits recognition accuracy compared to deep learning; evaluated only on German benchmark; no support for regional sign variants.
Zhang et al. (2021) [7]	Updated Chinese Traffic Sign Detection Benchmark (CTSDB) with 4,000+ real-world images annotated by sign type, size, and weather; benchmarked nine classic detection algorithms.	Focused exclusively on Chinese traffic signs; benchmark algorithms are classic (non-latest) deep learning methods; no real-time ADAS integration or voice alert component.

paper	Key Contributions	Limitations
Radojčić & Dobrojević (2025) [8]	Combined image enhancement and deep learning for robust TSR under complex backgrounds and varied environments; demonstrated strong potential for ADAS.	No specific dataset or benchmark details provided; regional road sign diversity (e.g., Arab countries) not addressed; no real-time or embedded deployment metrics reported.
Zhang et al. (2025) [9]	YOLOv8-based enhanced detection with small object detection layer and BiFPN for multi-scale detection; 90.1% mAP at 78 fps on TT100K, outperforming mainstream models.	Evaluated solely on TT100K (Chinese dataset); improvements focused on multi-scale detection without addressing regional sign variability or real-time driver assistance system integration.
Dagher & Makary (2025) [10]	Hybrid framework combining YOLOv12, Detectron2 (Faster R-CNN), and ViT classifier with overlapping box fusion; 97.7% mAP@0.5 on Roboflow dataset; strong on small, occluded, and overlapping signs.	High computational complexity from multi-model ensemble makes real-time edge deployment difficult; tested on a generic autonomous driving dataset without region-specific sign considerations.
Lim et al. (2023) [11]	Comprehensive survey covering preprocessing, feature extraction, classification, and datasets for TSR; identifies key challenges (geographic variability, illumination, complex backgrounds) and future directions.	Survey only, no new model, dataset, or system proposed; Middle Eastern and Arab road environments not specifically addressed in the geographic variability discussion.

The previous studies collectively reveal two recurring gaps: a lack of regional specificity, as most benchmarks and datasets are drawn from European, Chinese, or generic environments with little representation of Middle Eastern and locally specific traffic signs and road conditions, and limited system integration, where the majority of works evaluate detection and recognition in isolation without connecting model outputs to a practical driver assistance pipeline.

In contrast, this work addresses both by introducing a custom-built dataset collected from Jordanian streets using Google Street View [12], addressing the specific visual characteristics and sign conventions of the local road environment. Real-time traffic sign detection will be applied using the latest version of YOLO, and the system will further be integrated with a triggering mechanism that delivers a corresponding voice alert for each recognized sign, communicating the relevant road rule or warning to the driver in real time, bridging the gap between academic model performance and a deployable, user-facing driver assistance experience.

3. Methodology

3.1 Dataset

In this project we have constructed our own annotated dataset for detecting 25 distinct Jordanian traffic signs. Since Jordanian traffic signs are not part of a worldwide standard, 80% of the data was collected from Google Street View (Jordan), using the screen-shot app (Greenshot) for faster process, note that the images collected were taken from different distances and from many different Jordanian cities, while the remaining 20% consists of speed limit signs and few traffic lights sourced from an existing Roboflow [13] dataset, as these are visually similar to their Jordanian counterparts.



Fig 2. Samples from the Collected Data

Once the images were gathered, we proceeded with the annotation process using Label-Studio [14], a tool that enables manual bounding box drawing and object class assignment. Each image is paired with a corresponding .txt file containing the following format:

(Class ID X Y Width Height)

where X and Y represent the center coordinates of the bounding box, and width and height define its dimensions. This collecting & annotation process spanned approximately two weeks of work.

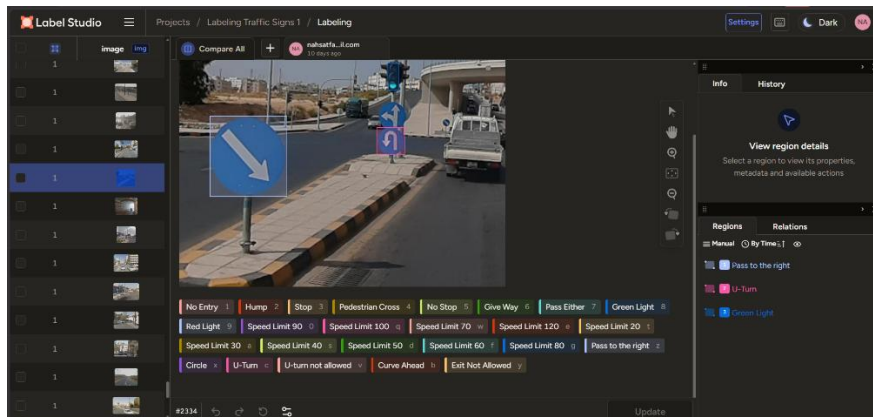


Fig 3. Label-Studio Interface



Fig 4. Images After Annotation

Following the collection and annotation stages, we performed dataset splits, dividing each dataset into training, validation, and test subsets. Augmentation techniques were then applied to the training sets, making copies with increased brightness, increased contrast, and horizontal flips, in order to improve model generalization, the library used for augmentation is Albumentations [15], as it automatically handles the bounding box annotations by generating a corresponding (.txt) file for each augmented image copy. lastly, we merged all images and their corresponding labels into one dataset.

Table 2: Dataset Numbers

Split	Annotated Images	Bounding Boxes
Train	20382	25262
Validation	2822	3311
Test	1793	2131

The split was performed using a stratified strategy to account for class imbalance, ensuring that each class, including underrepresented ones, is proportionally distributed across the training, validation, and test sets, so the model is exposed to and evaluated on all 25 classes.



Fig 5. Some Traffic Signs from Our Dataset

3.2 YOLO26 Architecture Overview

We selected YOLO26s from the Ultralytics framework [16], the small variant of the YOLO26 model family, officially released on January 14, 2026. YOLO26 was chosen over other YOLO versions for several compelling reasons that align well with our deployment goals.

YOLO26 is built around a native end-to-end, NMS-free architecture [17], predictions are generated directly without any non-maximum suppression post-processing step, a paradigm first pioneered in YOLOv10 [18] and now revisited and refined in YOLO26. This dramatically reduces inference latency and simplifies deployment. Complementing this, it also eliminates the Distribution Focal Loss (DFL) [19] module, which, while effective in earlier versions, complicated model export and limited hardware compatibility, its removal broadens support for edge and low-power devices.

On the training side, YOLO26 introduces the MuSGD optimizer, a hybrid of SGD [20] and Muon [21] inspired by Moonshot AI's Kimi K2 [22] LLM training breakthroughs. This brings improved stability and faster convergence by transferring optimization advances from language models into computer vision. Additionally, improved loss functions such as ProgLoss (progressive loss balancing) and STAL (small-target-aware label assignment) yield notably better small-object detection accuracy, a critical factor for our use case.

Collectively, these innovations make YOLO26 one of the most deployment-friendly YOLO releases to date, achieving up to 43% faster CPU inference compared to prior versions, making it well-suited for resource-constrained environments.

The algorithm follows a streamlined pipeline designed for real-time detection across both edge and server platforms. Input images are preprocessed and passed through a compact convolutional backbone that extracts multi-scale feature maps, preserving semantic information for objects of varying sizes. These maps are then fused through a lightweight neck before reaching the detection head, which directly outputs bounding boxes and class probabilities, eliminating the need for Non-Maximum Suppression entirely. Training is further stabilized by ProgLoss balancing and STAL assignment, which improve small-object detection, while the MuSGD optimizer ensures fast and reliable convergence. The architecture also supports FP16 and INT8 quantization, enabling efficient deployment across CPUs and GPUs with minimal accuracy loss.

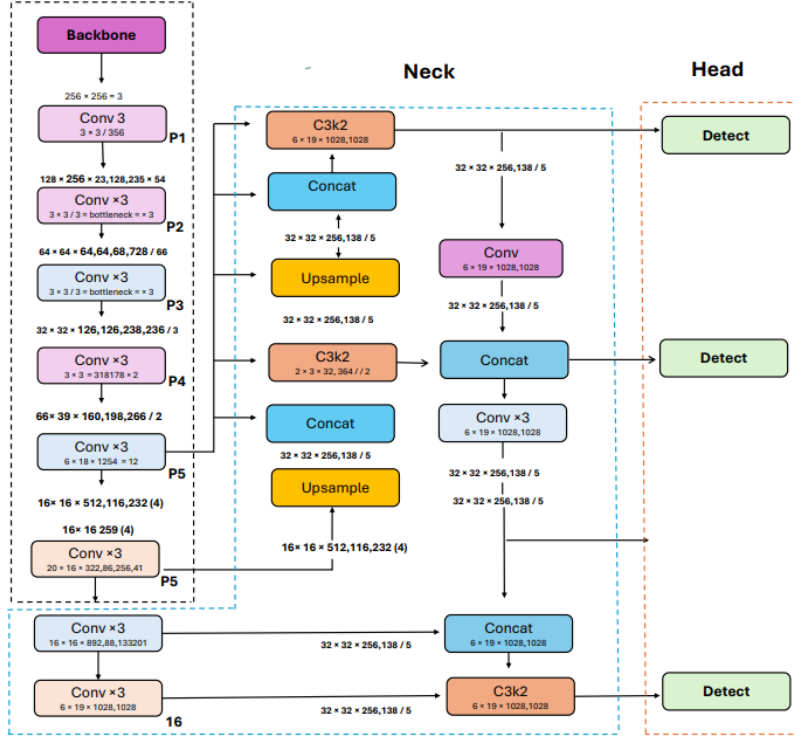


Fig 6. Core architecture diagram of Ultralytics YOLO26 object detection and segmentation algorithm

3.3 Model Training

Training was conducted on Google Colab [23] using a T4 GPU. The model was trained for 55 epochs on the full dataset at an image size of 640×640, with a batch size of 32 and Automatic Mixed Precision (AMP) enabled to optimize memory usage.

We used the MuSGD optimizer, YOLO26's newly introduced hybrid optimizer, with an initial learning rate of 0.01 and 3 warmup epochs. Since our dataset was already augmented during preparation, most built-in augmentations (color jitter, flipping, shear) were disabled to avoid over-augmenting, while mosaic augmentation was kept active and disabled only in the final 10 epochs as is standard practice.

The model was initialized from pretrained weights (yolo26s.pt) and trained with seed=42 for reproducibility. Results and checkpoints were saved directly to Google Drive [24] throughout training.

Due to Colab's session time limits, the runtime disconnected after approximately 7 hours at epoch 26. Training was seamlessly resumed from the last saved checkpoint in a second session, completing the remaining epochs without any loss of progress, in a total time of around 13 hours.



Fig 7. Training vs Validation losses

3.4 Model Deployment

Following training and evaluation, the model was prepared for real-world deployment as an assistive driver alert system. The deployment pipeline consisted of three main components: voice preparation, model export, and a real-time detection and announcement system.

To enable audio feedback, a set of Arabic voice announcements was recorded for each of the 25 traffic sign classes using an online text-to-speech tool [25] (TTS Maker). Each clip was kept concise, most under 2.5 seconds, to ensure announcements are delivered promptly without distracting the driver.

Table 3: Examples from The Voice Messages

Voice Message	Traffic Sign
احذَر، أَمَامَكَ مَطَبٌ	Hump
إِنْتَبِهْ، أَمَامَكَ مُنْعَطَفٌ	Curve Ahead
أَعْطِ الْأَوَّلِيَّةَ	Give Way

The trained model was then exported to the ONNX (Open Neural Network Exchange) format [26]. ONNX Runtime is a highly optimized inference engine that accelerates execution across CPUs, GPUs, and NPUs without requiring a full deep learning framework, making it well-suited for the target edge device which lacks powerful dedicated processors.

The core deployment script processes live video frames through the ONNX model and triggers voice announcements upon detection. Only detections exceeding a confidence threshold of 0.50 are considered, a value determined empirically during the real-world video evaluation to strike the best balance between sensitivity and false positive suppression. Since multiple signs can be detected

simultaneously or in rapid succession, playing all announcements at once would be chaotic and unsafe. To address this, detections are fed into a priority queue ordered by sign urgency, immediate danger signs such as Red Light, Stop, and No Entry are announced first, followed by cautionary signs like Give Way and Pedestrian Crossing, then restrictions, and finally informational signs such as speed limits. Two additional mechanisms prevent announcement fatigue: a cooldown period of 8 seconds suppresses repeated announcements of the same sign, and a staleness timeout of 5 seconds discards any queued detection that has waited too long, avoiding voice alerts that would play after the vehicle has already passed the sign.

4. Results and Analysis

To assess the performance of the trained YOLO26s model, three complementary evaluation approaches were conducted. The first is a standard quantitative evaluation on the held-out test set, measuring common computer vision metrics to objectively assess detection accuracy and reliability. The second is a real-world qualitative evaluation, where the model was tested on dashcam footage recorded from car rides through Jordanian streets, providing a practical assessment of how the model performs under real driving conditions, varying lighting, and local traffic sign appearances. The third is a full system evaluation conducted in real time, where the complete deployed system including the trained model, real-time inference pipeline, and voice alert mechanism, was run live during actual car rides across two Jordanian cities, Amman and Al-Tafila. This experiment assesses not only detection performance but also the responsiveness and practical usability of the system as a whole under authentic driving conditions. To assess the performance of the trained YOLO26s model, two complementary evaluation approaches were conducted. The first is a standard quantitative evaluation on the held-out test set, measuring common computer vision metrics to objectively assess detection accuracy and reliability. The second is a real-world qualitative evaluation, where the model was tested on dashcam footage recorded from car rides through Jordanian streets, providing a practical assessment of how the model performs under real driving conditions, varying lighting, and local traffic sign appearances.

4.1 Test Set Evaluation

The model was evaluated on the held-out test set of 1,793 images spanning all 25 classes. The key metrics used are mAP@50, mAP@50-95, Precision and Recall [27], where mAP@50 measures detection accuracy at an Intersection Over Union - IoU threshold of 50%, and mAP@50-95 provides a stricter aggregate score across multiple thresholds, giving a more comprehensive picture of localization quality.

The model achieved strong overall results:

Table 4: Results on The Test Set

Metric	Score
mAP@50	95.8%
mAP@50-95	84.6%
Precision	97.0%
Recall	95.6%

These results reflect a well-generalized model with high detection confidence and low miss rate across the majority of classes. Most classes performed consistently well, with several including Curve Ahead, No Entry, No Stop, Hump, Pedestrian Cross, and U-Turn, achieving mAP@50 scores above 99%.

The one notable underperformer is the Red-Light class, which recorded the lowest scores across all metrics (mAP@50: 78.5%, Recall: 72.7%) and also the Green Light with (mAP@50: 89.5%, Recall: 87.5%). This is likely attributable to the visual similarity between red and green traffic lights and the relatively high instance variability in lighting conditions, Red Light had 140 instances spread across only 92 images, with considerable appearance variation that the model found harder to consistently distinguish.

Figures 6 and 7 show two parts of the confusion matrix, providing an overview of the model’s performance. The remaining parts were omitted to improve clarity and readability.

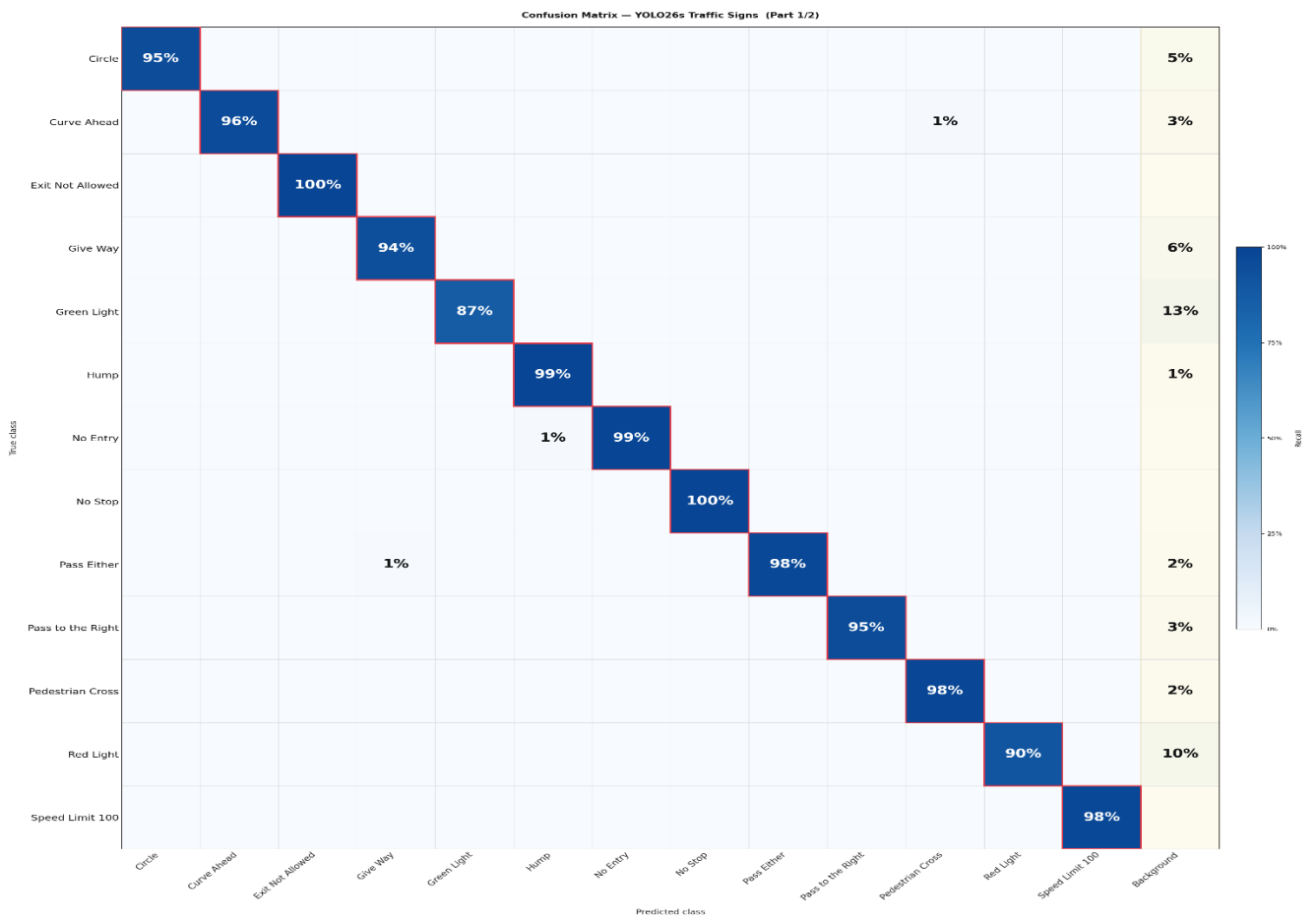


Fig 8. Confusion Matrix Results Part 1

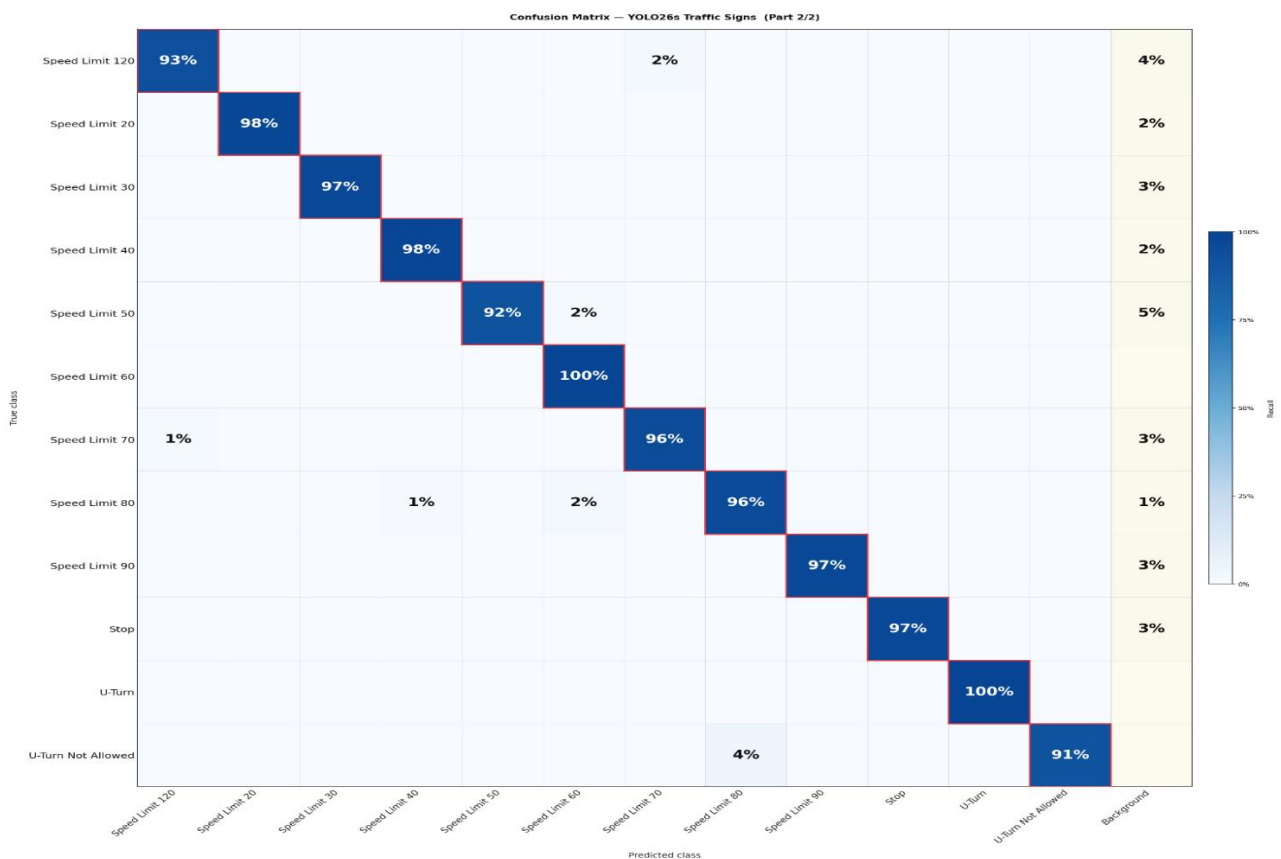


Fig 9: Confusion Matrix Results Part 2

4.2 Real-World Video Evaluation

The second evaluation involved testing the model on dashcam footage recorded from car rides through Jordanian streets, providing a practical measure of real-world performance beyond controlled test set conditions. The model demonstrated strong and consistent detection across varying road conditions, lighting, and sign appearances encountered in natural driving scenarios, with minimal false positives throughout. To determine the optimal operating point, the model was tested at several confidence thresholds. A threshold of 0.50 was found to strike the best balance, filtering out low-confidence spurious detections while retaining true positives reliably.

Notably, traffic light detection performed well in the video evaluation, which is encouraging given that the Red Light class was the weakest performer in the test set evaluation. This suggests that the model has learned meaningful visual features for traffic lights in practice, and that the test set metrics may partially reflect dataset-level factors rather than a fundamental limitation in the model's capability.



Fig 10. Model Evaluation on Video Footage

4.3 Real-Time Inference and Voice Alert System Evaluation

To evaluate the complete deployed system under authentic driving conditions, three real-world test drives were conducted across two Jordanian cities: one in Al-Tafilah and two in Amman. During each drive, the system ran live on a laptop (Intel Core i5-12450H), processing frames captured directly from the vehicle's dashcam feed in real time via the iVCam application [28], which streams a phone camera to the laptop as a virtual webcam. The YOLO26s model achieved approximately 10 FPS during CPU-only inference, which is sufficient for real-time traffic sign detection given that signs remain visible for several seconds under normal driving speeds. While a laptop is not the intended deployment target for such a system, its CPU-only inference performance provides a baseline frame rate sufficient to validate the real-time software logic, mimicking the practical operational speeds achieved by compact edge devices (such as a Raspberry Pi 5 with a Hailo-8L Kit or an NVIDIA Jetson Orin Nano), making it a valid stand-in for system validation purposes.

Unlike the previous evaluations, the criterion here is the voice alert behavior of the system, as the primary purpose of the deployment is to deliver timely and accurate Arabic spoken announcements to the driver. For each test drive, triggered voice alerts were assessed in terms of correctness, timeliness, and absence of disruptive false alerts. The results are summarized in Table 5 below.

Table 5: Real-Time Experiment Results

Experiment	Duration	Traffic Sign types	Appeared	Captured (TP)	False Alerts (FP)
Al-Tafila	20 mins	11	46	45	5
Amman (route 1)	30 mins	16	83	83	9
Amman (route 2)	33 mins	18	113	109	10

The results demonstrate strong real-time performance of the deployed system under different driving environments and conditions. Across the three experiments, the system successfully detected nearly all encountered traffic signs, with only a very small number of missed detections, indicating highly reliable real-time inference and alert generation. In total, 19 unique traffic sign classes were encountered during the experiments, covering a diverse set of road scenarios. Most false alerts occurred when the model encountered traffic signs or road symbols that were not included in the training dataset, causing them to

be incorrectly classified as known classes. The experiments were conducted under varying conditions, including differences in traffic density, road structure, and time of day, yet the system maintained stable performance with very few false negatives. These results show that the proposed system can operate effectively in practical driving scenarios while providing timely and reliable Arabic voice alerts to the driver.

5. Conclusions and Recommendations

This project presented the design and implementation of a real-time traffic sign detection and Arabic voice alert system tailored for local driving environments in Jordan. The obtained results demonstrate that the proposed system achieves strong detection performance and reliable real-time operation under practical road conditions, with very few missed detections and a low number of false alerts. The deployment experiments conducted across different streets, traffic conditions, and times of day confirmed the robustness and stability of the system, showing its ability to provide timely Arabic spoken alerts that can assist drivers during everyday driving scenarios. Overall, the system proved to be an effective and practical solution that performs well on local streets using lightweight hardware and real-time inference.

For future work, the dataset can be further expanded to include additional traffic sign categories and a larger variety of samples for existing classes under different lighting, weather, and road conditions. This would improve the model's generalization capability and help reduce false positive detections caused by unseen or visually similar signs. Future improvements may also include deploying the system on dedicated embedded AI hardware, optimizing inference speed, and integrating additional driver assistance features to further enhance real-world usability and reliability.

Moreover, the developed YOLO-based traffic sign detection model is not limited to the Arabic voice alert application presented in this project. The model can serve as a foundation for integration into broader intelligent transportation and driver assistance systems, including autonomous vehicles and smart road monitoring applications. It may also contribute to future efforts aimed at automating driving tests in Jordan, aligning with the vision of the Driver and Vehicle Licensing Department - Jordan to modernize and improve driving evaluation procedures through technology. Ultimately, the main goal of this project is to contribute toward safer roads and improved driver awareness, and it is hoped that this work can one day be further developed and utilized to help make streets safer for everyone.

Reference list

- [1] Sapkota, R., Cheppally, R. H., Sharda, A., & Karkee, M. (2025). YOLO26: key architectural enhancements and performance benchmarking for real-time object detection. *arXiv preprint arXiv:2509.25164*.
- [2] Mathias, Markus, et al. "Traffic sign recognition—How far are we from the solution?." *The 2013 international joint conference on Neural networks (IJCNN)*. IEEE, 2013.
- [3] Zhu, Yanzhao, and Wei Qi Yan. "Traffic sign recognition based on deep learning." *Multimedia Tools and Applications* 81.13 (2022): 17779-17791.
- [4] Alghmgham, Danyah A., et al. "Autonomous traffic sign (ATSR) detection and recognition using deep CNN." *Procedia Computer Science* 163 (2019): 266-274.
- [5] Zheng, Yuping, and Weiwei Jiang. "Evaluation of vision transformers for traffic sign classification." *Wireless Communications and Mobile Computing* 2022.1 (2022): 3041117.
- [6] Weng, Huai-Mao, and Ching-Te Chiu. "Resource efficient hardware implementation for real-time traffic sign recognition." *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2018.
- [7] Zhang, Jianming, et al. "CCTSDb 2021: A more comprehensive traffic sign detection benchmark." *Human-centric Computing and Information Sciences* 12 (2022).
- [8] Radojčić, Vesna, and Miloš Dobrojević. "Traffic Sign Recognition Using Computer Vision." *Sinteza 2025-International Scientific Conference on Information Technology, Computer Science, and Data Science*. Singidunum University, 2025.
- [9] Zhang, Hong, Mingyin Liang, and Yufeng Wang. "YOLO-BS: a traffic sign detection algorithm based on YOLOv8." *Scientific Reports* 15.1 (2025): 7558.
- [10] Dagher, Issam, and Antonio Makary. "Detection Framework for Traffic Signs Using a Hybrid Approach." *IEEE Access* (2025).
- [11] Lim, Xin Roy, et al. "Recent advances in traffic sign recognition: approaches and datasets." *Sensors* 23.10 (2023): 4674.
- [12] Anguelov, Dragomir, et al. "Google street view: Capturing the world at street level." *Computer* 43.6 (2010): 32-38.
- [13] Lin, Qinjie, et al. "Roboflow: a data-centric workflow management system for developing ai-enhanced robots." *Conference on Robot Learning*. PMLR, 2022.
- [14] JAKUBCIK, MATE J. "Enhancements of the Label Studio Annotation Tool."
- [15] Buslaev, Alexander, et al. "Albumentations: fast and flexible image augmentations." *Information* 11.2 (2020): 125.
- [16] Sapkota, Ranjan, and Manoj Karkee. "Ultralytics YOLO evolution: An overview of YOLO26, YOLO11, YOLOv8 and YOLOv5 object detectors for computer vision and pattern recognition." *arXiv preprint arXiv:2510.09653* (2025).
- [17] Si, King-Siong, et al. "Accelerating non-maximum suppression: a graph theory perspective." *Advances in Neural Information Processing Systems* 37 (2024): 121992-122028.
- [18] Wang, Ao, et al. "Yolov10: Real-time end-to-end object detection." *Advances in neural information processing systems* 37 (2024): 107984-108011.
- [19] Li, Xiang, et al. "Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection." *Advances in neural information processing systems* 33 (2020): 21002-21012.
- [20] Ketkar, Nikhil. "Stochastic gradient descent." *Deep learning with Python: A hands-on introduction*. Berkeley, CA: Apress, 2017. 113-132.
- [21] Shen, Wei, et al. "On the convergence analysis of muon." *arXiv preprint arXiv:2505.23737* (2025).
- [22] Team, Kimi, et al. "Kimi k2: Open agentic intelligence." *arXiv preprint arXiv:2507.20534* (2025).
- [23] Bisong, Ekaba. "Google colab." *Building machine learning and deep learning models on google cloud platform: a comprehensive guide for beginners*. Berkeley, CA: Apress, 2019. 59-64.
- [24] Gallaway, Teri Oaks, and Jennifer Starkey. "Google drive." *The Charleston Advisor* 14.3 (2013): 16-19.
- [25] Kumar, Yogesh, Apeksha Koul, and Chamkaur Singh. "A deep learning approaches in text-to-speech system: a systematic review and recent research perspective." *Multimedia Tools and Applications* 82.10 (2023): 15171-15197.
- [26] Bai, J., Lu, F., Zhang, K. et al.: ONNX: Open Neural Network Exchange, GitHub (online), available from <https://github.com/onnx/onnx> (accessed 2020-07-01).

- [27] Padilla, Rafael, Sergio L. Netto, and Eduardo AB Da Silva. "A survey on performance metrics for object-detection algorithms." *2020 international conference on systems, signals and image processing (IWSSIP)*. IEEE, 2020.
- [28] Crandall, Logan, Mason Roberts, and En Cheng. "Mimic: a remote webcam device over webrtc." *2021 International Conference on Computational Science and Computational Intelligence (CSCI)*. IEEE, 2021.